Weekly Review 106

De Morgan's Law

Keywords: Commutative; Associative; Distributive; Identity; De Morgan's Law;

Starter: Boolean Algebra

In pairs, write down as many properties and identities of Boolean Algebra as you can remember.

Hint: There are 3 named properties, each with 2 equations, and 8 boolean identities, 4 for each main gate.

Starter: Boolean Algebra

Commutative	$A.B = B.A$ $A + B = B + A$
Associative	$A.(B.C) = (A.B).C$ $A + (B + C) = (A + B) + C$
Distributive	$A.(B + C) = A.B + A.C$ $A + B.C = (A + B).(A + C)$
AND	$A \cdot 0 = 0, \quad A \cdot 1 = A,$ $A \cdot A = A, \quad A \cdot \overline{A} = 0$
OR	$A + 0 = A, \quad A + 1 = 1,$ $A + A = A, \quad A + \overline{A} = 1$

Simplifying Boolean

How to simplify Boolean Algebra

1. Look at the expression
2. Don't panic
3. Use one of the identities or properties of commutativity, associativity, distributivity to reduce one part of the expression
4. Repeat step 3 until there is nothing left to reduce

You Try

1. $B \cdot C + B \cdot \overline{C}$

2. $A + (A \cdot C)$

3. $\overline{A} \cdot \overline{B} + \overline{A} \cdot B + A \cdot \overline{B} + A \cdot B$

4. $\overline{A} \cdot \left(B \cdot \overline{C} + C \cdot B\right) + \overline{C} \cdot B \cdot A + B \cdot A \cdot C$

5. $\overline{A} \cdot \left(B \cdot \overline{C} + (B + A) \cdot \left(B + \overline{A}\right) \cdot C\right) + A \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot C$

Complete the truth tables for:

$\overline{A \cdot B}$

A	B	$A \cdot B$	$\overline{A \cdot B}$
0	0		
0	1		
1	0		
1	1		

$\overline{A + B}$

A	B	$A + B$	$\overline{A + B}$
0	0		
0	1		
1	0		
1	1		

Complete the truth tables for:

$\overline{A \cdot B}$

A	B	$A \cdot B$	$\overline{A \cdot B}$
0	0	0	
0	1	0	
1	0	0	
1	1	1	

$\overline{A + B}$

A	B	$A + B$	$\overline{A + B}$
0	0	0	
0	1	1	
1	0	1	
1	1	1	

Complete the truth tables for:

$\overline{A \cdot B}$

A	B	$A \cdot B$	$\overline{A \cdot B}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

$\overline{A + B}$

A	B	$A + B$	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Those are logic gates we already have!

De Morgan's Law states:

$$\overline{A \cdot B} \equiv \overline{A} + \overline{B}$$

and

$$\overline{A + B} \equiv \overline{A} \cdot \overline{B}$$

We often say:
"Break the line, change the sign"

Those are logic gates we already have!

De Morgan's Law states:

$$\overline{A \cdot B} \equiv \overline{A} + \overline{B}$$

and

$$\overline{A + B} \equiv \overline{A} \cdot \overline{B}$$

We often say:
"Break the line, change the sign"

That's much easier than Demorgan's "Negating both terms separately, then negating the whole expression and change the sign between them is logically complete"

Example Time!

Let's simplify:

$$(A \cdot B) \cdot \overline{\overline{A} + \overline{B}}$$

You try:

1. $\overline{(A + B) \cdot C}$

2. $\overline{A + B \cdot C}$

3. $\overline{\overline{\overline{A} \cdot B} + C}$

4. $\overline{A \cdot B} + A$

5. $\overline{\overline{\overline{A} + B} + B}$

6. $\overline{\overline{B} + \overline{A + B}}$

Structures

Keywords: Structure; Composite Datatype; static memory allocation;

Variables vs Structures

Age: Integer

Name: String

Price: Integer

Alive: Boolean

We can store information in variables!

Variables vs Structures

Age: Integer

Name: String

Price: Integer

Alive: Boolean

We can store information in variables!

What if we wanted to store lots of information in one place?

Variables vs Structures

Age: Integer

Name: String

Price: Integer

Alive: Boolean

We can store information in variables!

What if we wanted to store lots of information in one place?

Structures!

Car
Price: Integer
Owner: String
Age: Integer

Variables vs Structures

Age: Integer

Name: String

Price: Integer

Alive: Boolean

We can store information in variables!

What if we wanted to store lots of information in one place?

Structures!

Car
Price: Integer
Owner: String
Age: Integer

Player
Position: Coord
Shape: Char
Alive: Boolean

Variables vs Structures

Age: Integer

Name: String

Price: Integer

Alive: Boolean

We can store information in variables!

What if we wanted to store lots of information in one place?

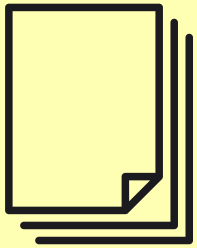
Structures!

Car
Price: Integer
Owner: String
Age: Integer

Coord
X: Integer
Y: Integer

Player
Position: Coord
Shape: Char
Alive: Boolean

Structures



- Structures are like containers
- They group together a set of variables that are related to create a **record**
- Structures are **statically allocated**

```
struct Book {
  public string title;
  public string author;
  public string genre;
  public decimal price;
}
```

This is the template for a Book structure.

Example

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Example

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Structures define templates

Example

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Structures define templates

Then you can declare variables of that type.

Example

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Structures define templates

Then you can declare variables of that type.

You access the data with the '.'

You Try

- Create a structure called **Person**.
- It should hold their: title, full name, age and favourite colour.
- In `Main()`, create a **Person** variable called `myself`, and store the information about yourself.
- Write a procedure called `DisplayPerson`, which takes a **Person** as a parameter, and prints out the information about the person in a useful format.

EXT: Write a function called `InputPerson` that asks for the person's information, and returns a new **Person** object. Call this in `Main`, and pass the result into `DisplayPerson`

What makes a good discussion?

Comp Sci involves a lot of discussion in lessons.

What is the benefit of discussing ideas in lessons?

What features does an effective discussion have?

What makes a discussion ineffective?

PRIMM task!

Find the PRIMM task on **GClassroom** to help you practice structures.

Consider:
Why do we use PRIMM, even if you're confident enough to go straight to the Make tasks?